

第12章 注解与编译期配置

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:熟悉 C 的 `#ifdef/#ifndef`、`__attribute__`、`-Wall/-Werror` 与 Makefile/CMake 的条件开关

本章学习目标

- 理解属性(attribute)在 Rust 中的角色: C 的预处理指令 + `__attribute__` + 编译器选项,三者合一
- 掌握属性语法与分类:内置属性按用途分组,能认出与检索
- 掌握 `#[cfg]` 条件编译,与 C 的 `#ifdef/#ifndef` 逐点对照,并掌握 `cfg!()` 运行期判断
- 掌握 `#[derive]` 的常用组合与适用场景(承接第10章的原理)
- 掌握 lint 系统:`#[allow]`/`#[deny]`/`#[warn]` 与 C 的 `-Wall/-Werror` 对照,理解 `#[must_use]`
- 了解自定义属性宏(过程宏 attribute 类):`#[timed]` 式改写函数的原理与写法
- 掌握 Cargo features 特性开关:编译期配置的工程标准(对照 CMake 的选项)
- 能在"运行期配置"与"编译期配置"之间做出正确选择

从 C 到 Rust:本章主题如何迁移

C 的"编译期配置"散落在三个世界:**预处理指令** `#ifdef/#ifndef/#define`(文本级条件)、**编译器属性** `__attribute__((...))`(给编译器的提示,各编译器方言不一)、**编译器选项** `-Wall/-Werror`/宏定义。三者语法不同、作用时机不同、跨平台表现不同:`#ifdef` 是文本替换,能破坏语法(`#define bool int` 的灾难);`__attribute__` 在 GCC 与 MSVC 之间不通用;警告开关换个编译器就失效。

Rust 把这三者统一成一套**属性系统**:以 `#[...]` 形式附着在代码条目上,由编译器在语法解析后统一处理。`#[cfg]` 是类型安全的条件编译(不像 `#ifdef` 那样文本替换);`#[allow]`/`#[deny]` 是每个 crate 可配的 lint 开关;`#[derive]` 是代码生成;自定义属性宏把"编译器扩展"开放给库作者。配套的 Cargo features 提供工程级的"构建选项"——比 CMake 的 `option()` 更结构化(类型化、可组合、可推导)。本章的路线:先看 C 的三个世界如何对应过来(\$12.1-12.2),再逐个掌握 `#[cfg]`(\$12.3)、`#[derive]`(\$12.4)、lint 系统(\$12.5),然后是属性宏(\$12.6)与 features(\$12.7)。

前置知识自查

- C 的 `#ifdef/#ifndef/#else/#endif`、`#define` 宏、`__attribute__((unused/format))`、`-Wall -Werror`
- 第10章的宏与 `derive` 原理(属性宏是过程宏的一种)
- 第11章的 `serde` 属性用法(`#[serde(rename)]`)——你已经用过属性了
- Cargo.toml 的基本结构(第13章会系统讲,本章先用 `features`)
- (可选)Makefile/CMake 的开关变量与条件编译

本章学习路线

1. **第一阶段(约1小时):**§12.1–§12.2,属性系统总览与语法——先建立"三合一"的心智模型
2. **第二阶段(约1.5小时):**§12.3–§12.5,#[cfg]/#[derive]/lint——日常代码里出现频率最高的三个属性族
3. **第三阶段(约1.5小时):**§12.6–§12.7,属性宏与 features——生产级编译期配置的完整形态
4. **验收标准:**能徒手写"平台分支 + 特性开关 + 测试打印开关"三合一的条件编译代码,并说出每个属性在编译期做什么

本章知识导图

- §12.1 C 的三个世界 vs Rust 的属性系统
- §12.2 属性语法:位置、参数与分类
- §12.3 #[cfg] 条件编译与 cfg!()
- §12.4 #[derive] 派生宏的常用组合
- §12.5 lint 系统:#[allow]/#[deny]/#[must_use]
- §12.6 自定义属性宏:#[timed] 的原理
- §12.7 Cargo features:工程级编译开关
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§12.1 C 的三个世界 vs Rust 的属性系统

12.1.1 C 的条件编译:文本级的危险艺术

```
// C:预处理在语法解析之前,条件编译是"文本删减"
#ifdef _WIN32
    #include <windows.h>
    #define PATH_SEP '\\\'
#elif defined(__linux__)
    #include <unistd.h>
    #define PATH_SEP '/'
#else
    #error "不支持的平台"
#endif

// 风险:被 #ifdef 删掉的内容不参与语法检查——错误藏在"没编译到的分支"里
#ifdef DEBUG
    printf("debug=%d\n", dbg);    // 拼错变量名?只有开 DEBUG 才炸
#endif
```

12.1.2 Rust:属性由编译器统一处理

```
// Rust:条件编译在语法解析之后—所有分支都先过语法检查,再决定取舍
#[cfg(target_os = "windows")]
fn path_sep() -> char { '\\' }

#[cfg(target_os = "linux")]
fn path_sep() -> char { '/' }

// 关键差异:#ifdef 里写错语法没人管;
// #[cfg] 里两个分支都必须语法正确,只是"类型检查与代码生成"按条件进行
fn main() {
    println!("分隔符:{}", path_sep());
}
```

C 世界	Rust 对应	差异要点
#ifdef / #ifndef / #endif	#[cfg(...)] / #[cfg(not(...))]	Rust 先解析语法再条件取舍;C 先删文本再解析
#define 编译期常量	const / cfg 选项 / features	无文本替换,类型安全
__attribute__((...))	#[...] 属性	统一语法,跨平台一致
-Wall / -Werror / -Wno-xxx	#[warn]/#[deny]/#[allow] Cargo.lints	+ 配置进源码,随 crate 分发
CMake option() 开关	Cargo.toml [features]	类型化、可组合、可被依赖方覆盖
#pragma once / include 守卫	模块系统(crate/mod,第13章)	无头文件,无重复包含问题

§12.2 属性语法:位置、参数与分类

12.2.1 语法形态

```
// 三种写法:
// 1) 无参数:作用于紧随其后的条目
#[derive(Debug)]
struct Point { x: f64, y: f64 }

// 2) 有参数:括号内是参数
#[cfg(target_os = "windows")]
fn win_only() {}

// 3) 多个属性叠放
#[derive(Clone, PartialEq)]
#[allow(dead_code)]
struct Config { port: u16 }

// 4) 作用于整个文件/模块的"内部属性"(!)
#![allow(unused_variables)] // 放在 crate/模块最顶部,作用于整个容器
fn main() {
    let unused = 42; // 不再警告
    println!("ok");
}
```

12.2.2 内置属性的六大用途分组

用途	代表属性	C 对应
条件编译	<code>#[cfg]</code> / <code>#[cfg_attr]</code>	<code>#ifdef</code> / <code>#if</code>
代码生成	<code>#[derive]</code> / <code>#[test]</code>	宏 / 无
lint 控制	<code>#[allow]</code> / <code>#[deny]</code> / <code>#[warn]</code> / <code>#[must_use]</code>	<code>-Wno-xxx</code> / <code>-Werror</code>
入口与外部接口	<code>#[main]</code> / <code>#[no_mangle]</code> / <code>#[repr]</code> / <code>#[link]</code>	<code>extern "C"</code> / 链接选项
布局与 ABI	<code>#[repr(C)]</code> / <code>#[repr(u8)]</code> / <code>#[repr(transparent)]</code>	结构体布局约定
静态保证	<code>#[inline]</code> / <code>#[cold]</code> / <code>#[deprecated]</code>	<code>__attribute__((always_inline))</code>

§12.3 #[cfg] 条件编译与 cfg!()

12.3.1 平台分支的正确姿势

```
// 平台相关代码:cfg 表达式是"真布尔逻辑",支持 and/or/not
#[cfg(all(target_os = "linux", target_arch = "x86_64"))]
fn platform_desc() -> &'static str { "Linux x86_64" }

#[cfg(any(target_os = "windows", target_os = "macos"))]
fn platform_desc() -> &'static str { "Windows 或 macOS" }

#[cfg(not(debug_assertions))]
fn build_profile() -> &'static str { "release" }

#[cfg(debug_assertions)]
fn build_profile() -> &'static str { "debug" }

fn main() {
    println!("{}", platform_desc(), build_profile());
}
```

12.3.2 cfg!():运行期布尔,不是删代码

```
// cfg!() 在编译期求值为 true/false 常量,代码始终存在:
// 适合"分支很小、不值得物理删除"的场景
fn main() {
    if cfg!(debug_assertions) {
        println!("调试构建:启用详细日志");
    } else {
        println!("发布构建:日志降级");
    }

    // 对照 C:#ifdef DEBUG + printf 的成对出现;
    // cfg!() 让"环境判断"变成普通布尔表达式,可在任何表达式位置用
    let threshold: u32 = if cfg!(target_pointer_width = "64") { 1_000_000 } else { 100_000 };
    println!("阈值:{threshold}");
}
```

⚠ 易错点 1:#[cfg] 删条目,cfg!() 保留代码——别混用

#[cfg] 作用于条目时,不匹配的条目 **整个不编译**:其中的类型错误、未定义名称都不会报错(对照 #ifdef 的风险,但语法仍检查);cfg!() 是"编译期常量布尔",代码必须始终合法。判据:整个函数/模块要不要存在→#[cfg];一段逻辑的取值差异→cfg!()。最典型的混用错误:给函数挂 #[cfg(feature = "x")],却在别处无条件调用它——未启用特性时编译错误"function not found",这正是设计意图,也是查错的方向。

12.3.3 cfg_attr:条件化地加属性

```
// #[cfg_attr(条件, 属性)]:条件成立时才附加属性
// 生产场景:release 下为热路径函数附加内联提示
#[cfg_attr(not(debug_assertions), inline)]
fn hot_add(a: u32, b: u32) -> u32 { a + b }

// 或:只有特定平台才派生某 trait
#[cfg_attr(target_os = "windows", derive(Debug))]
struct WinInfo { name: String }

fn main() {
    println!("{}", hot_add(1, 2));
}
```

§12.4 #[derive] 派生宏的常用组合

```
// 生产代码的标准组合(第10章讲原理,这里讲"选型")
#[derive(Debug, Clone, PartialEq)]
struct Task { id: u32, name: String }

#[derive(Debug, Clone, Copy, PartialEq)]
struct Vec2 { x: f64, y: f64 }          // 小值类型:加 Copy

#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct Key { a: u32, b: u32 }          // 作 HashMap 键:Eq + Hash

fn main() {
    let t1 = Task { id: 1, name: "a".into() };
    let t2 = t1.clone();                // Clone
    assert_eq!(t1, t2);                 // PartialEq
    println!("{:?}", t2);               // Debug

    let v = Vec2 { x: 1.0, y: 2.0 };
    let v2 = v;                         // Copy:隐式拷贝,不用 clone
    println!("{:?} {:?}", v, v2);
}
```

派生 trait	生成能力	适用类型
Debug	{:?} 结构化打印	几乎所有类型(日志/错误链)
Clone	显式深拷贝 .clone()	需要复制的类型
Copy	隐式按位拷贝(值语义)	小值类型:数字、坐标、标志
PartialEq	== / !=	需要比较的类型
Eq + Hash	可作 HashMap/HashSet 键	键类型
PartialOrd / Ord	< > 与排序	需要排序/取极值的类型
Default	default() 零值构造	配置类、可空字段容器

§12.5 lint 系统:#[allow]/#[deny]/#[must_use]

12.5.1 三档级别与 C 的警告开关

```
// 三档:allow(允许)/ warn(警告,默认)/ deny(升级为错误)
#![deny(unsafe_code)]          // 整个 crate 禁止 unsafe(对照 -Werror 的极端版)
#![deny(missing_docs)]         // 公共 API 缺文档即编译失败(生产库的标配)

#![allow(dead_code)]           // 局部豁免:这个结构体暂时没用,别警告
struct Wip { x: u32 }

#![warn(unused_variables)]      // 显式声明某条 lint 的级别(一般用不到,默认已开)

fn main() {
    let tmp = 42;                // 默认 warn:unused_variables
    println!("看,没有警告");
    let _ = tmp;
}
```

12.5.2 #[must_use]:防止"丢了返回值"

```
// C 的经典事故:fclose/strtok/printf 的返回值没人查
// Rust 用 #[must_use] 把"必须使用返回值"写进类型契约

#![must_use = "校验失败会被静默吞掉,务必处理 Result"]
fn validate(input: &str) -> Result<(), String> {
    if input.is_empty() {
        Err("输入为空".into())
    } else {
        Ok(())
    }
}

fn main() {
    // validate("x");           // 警告:unused must_use value
    let _ = validate("x");      // 显式忽略:警告消失(有意为之)

    // 标准库里 #[must_use] 的著名成员:Result、MutexGuard、Iterator
    let v = vec![1, 2, 3];
    let _iter = v.iter().map(|x| x * 2); // 不收集会警告吗?不会——map 是惰性的,
    // 但"算完没保存"的典型错误会警告:std 已把该标的都标了
    println!("{}", validate("x").is_ok());
}
```

工程建议:新项目的 lint 起点

Cargo.toml 里配 `[lints] rust = { unsafe_code = "deny", missing_docs = "warn" }`,把策略随 crate 版本化(第13章 Cargo 全流程会细讲)。从 C 转来的程序员最值得先开的两条:`unsafe_code`(让 `unsafe` 边界显式化)与 `clippy` 全家桶(`clippy --fix` 自动修)——这相当于把 `-Wall -Wextra -Werror` 变成默认,并且更智能。

§12.6 自定义属性宏:#[timed] 的原理

12.6.1 需求:给函数自动加计时

```
// 想写这样:任何函数挂上 #[timed],调用时自动打印耗时
// #[timed]
// fn work() { ... }

// 属性宏的职责:读取被标注的函数,在其外层生成"计时包裹代码"。
// 过程宏必须放在独立的 proc-macro crate(设计如此:宏是编译器的插件)
```

12.6.2 完整的最小实现(教学简化版)

```
// ---- timer-macros/src/lib.rs(独立 crate 的完整源码)----
// Cargo.toml 关键段:
// [lib] proc-macro = true
// [dependencies] quote = "1", syn = "2"

use proc_macro::TokenStream;
use quote::quote;

/// #[timed] 属性宏:包裹无参函数,调用时打印耗时
/// 教学简化版:只处理无参数、无返回值的 fn
#[proc_macro_attribute]
pub fn timed(_attr: TokenStream, item: TokenStream) -> TokenStream {
    // 1. 把被标注的条目解析为"函数"AST
    let func = syn::parse_macro_input!(item as syn::ItemFn);
    let name = &func.sig.ident;
    let body = &func.block;

    // 2. 生成新的函数体:原文被包进"计时包裹"
    let output = quote! {
        fn #name() {
            let __start = std::time::Instant::now();
            #body
            println!("[计时] {} 耗时 {:?}",
                    stringify!(&name), __start.elapsed());
        }
    };

    // 3. 输出替换原条目的代码(编译期完成)
    output.into()
}
```



```
// ---- 使用方 main.rs ----
use timer_macros::timed;

#[timed]                                // 宏在编译期改写这个函数
fn work() {
    let mut sum = 0u64;
    for i in 1..=1_000_000 {
        sum += i;
    }
    println!("sum = {sum}");
}

fn main() {
    work();                               // [计时] work 耗时 1.2ms
    work();                               // 每次调用都自动计时
}
```

⚠ 易错点 2:过程宏是"编译器插件",有隔离与版本风险

过程宏 crate 用独立的编译器实例运行,不能与主 crate 共享代码;宏代码里的 panic 会变成编译错误(信息难看)。生产上:优先用成熟宏库(tracing、thiserror、serde),自研属性宏只在你确实需要"改写代码形态"时做——写好文档、做好测试(宏展开测试用 trybuild)。用 cargo expand 预览展开结果,是调试宏的第一工具。

§12.7 Cargo features:工程级编译开关

12.7.1 features 的声明与使用

```
# ---- Cargo.toml(工程配置) ----
# [features] 表:每个特性是一个"开关",可以被默认启用或被依赖方按需打开
#
# [features]
# default = ["std"]           # 默认启用的特性列表
# std = []                    # 空特性:纯粹的标志
# verbose = ["std"]           # 特性可以依赖其他特性
#
# [dependencies]
# serde = { version = "1", optional = true } # 可选依赖,随特性启用
```

```
// ---- src/main.rs(代码侧) ----
// 特性开关在代码里通过 #[cfg] 感知

#[cfg(feature = "std")]
fn io_support() -> &'static str { "标准库可用" }

#[cfg(not(feature = "std"))]
fn io_support() -> &'static str { "no_std 环境" }

// 可选依赖的惯用法:特性名与依赖同名,启用特性才编译依赖
#[cfg(feature = "verbose")]
fn log_detail(msg: &str) {
    println!("[详细] {msg}");
}

#[cfg(not(feature = "verbose"))]
fn log_detail(_msg: &str) {
    // 无操作:特性关闭时调用方代码不变,行为降级
}

fn main() {
    println!("{}", io_support());
    log_detail("处理完成");
}
```

12.7.2 与 CMake 的对照

CMake 概念	Cargo features
option(VERBOSE "... OFF) + if(VERBOSE)	[features] verbose = [] + #[cfg(feature = "verbose")]
add_definitions(-DVERBOSE)	cargo build --features verbose
target_compile_definitions 按目标	#[cfg(target_os = ...)] 平台条件
find_package 可选依赖	[dependencies] xxx = { optional = true }
开关变量作用域(目录级)	特性随 crate 传递:依赖方可覆盖
配置漂移(忘开/错开)	编译期 #[cfg] 校验,未定义特性名编译报错

★ 重点:运行期配置 vs 编译期配置,怎么选

同一份"配置",两条路:运行期配置(env/配置文件,第7章)支持热切换、用户可调;编译期配置(features/#[cfg])减小体积、去掉攻击面、让代码分支成为死代码。判据:值在构建时确定且不需要切换(平台、no_std、嵌入大小)→编译期;运行中可变或用户可调(端口、日志级别、功能开关)→运行期。生产系统常混合:编译期定"支持哪些能力",运行期定"当前启用哪些"。C 程序员要警惕的惯性:#ifdef 开太多会把产品拆成一堆不可测的组合——features 组合爆炸同样存在,用"default 合理 + 文档 + CI 矩阵"控制。

⚠ 易错点 3:feature 名拼写错误是"静默关特性"

Cargo 构建时,未知的 feature 名会报错;但代码里的 `#[cfg(feature = "verbose")]` 与 Cargo.toml 的声明必须严格一致——改名后漏改一处,特性就"静默关闭"(不报错,只是代码不编译进二进制)。对策:CI 里至少跑一次 `--all-features` 构建,把"所有特性全开"的形态纳入验证;这正是 C 的 `#ifdef` 组合爆炸问题在 Rust 里的对应管理手段。

把"运行期配置"与"编译期配置"放进同一套代码,是生产系统的常态:features 决定"这份二进制支持什么"(后端类型、可选依赖),运行期配置决定"这次运行怎么用"(端口、级别、开关)。C 程序员最容易犯的惯性是把一切做成 `#ifdef`——结果产品被拆成几十种互不相同的二进制,测试矩阵爆炸。Rust 的纪律:编译期开关必须数量少、语义正交(default 合理);一旦出现"两个特性必须同时开"的隐式耦合,说明该特性应该合并,或改为运行期配置。特性是"能力",不是"模式"——这个心智模型,是生产级编译期配置的总纲。

本章速查表

主题	结论 / 写法
属性本质	<code>#[...]</code> 附着条目,编译器语法解析后处理;替代 <code>#ifdef + __attribute__ + 编译选项</code>
条件编译	<code>#[cfg(target_os = "windows")]</code> / <code>#[cfg(not(debug_assertions))]</code>
cfg 组合	<code>all(...)</code> / <code>any(...)</code> / <code>not(...)</code> 布尔组合
运行期判断	<code>cfg!(debug_assertions)</code> 编译期常量布尔,代码始终存在
条件属性	<code>#[cfg_attr(cond, attr)]</code> :条件成立才附加属性
代码生成	<code>#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]</code>
lint 三档	<code>#[allow(x)]</code> 放行 / <code>#[warn(x)]</code> 警告 / <code>#[deny(x)]</code> 升级错误
crate 级 lint	<code>#![deny(unsafe_code)]</code> 顶部内部属性,作用于整个 crate
必须使用	<code>#[must_use]</code> 标记类型/函数:忽略返回值即警告
属性宏	<code>#[proc_macro_attribute]</code> 改写被标注条目;独立 proc-macro crate + <code>syn/quote</code>
特性声明	Cargo.toml [features] default = ["std"];verbose = []
特性感知	<code>#[cfg(feature = "verbose")]</code> 与 <code>#[cfg(not(...))]</code> 双分支
可选依赖	[dependencies] serde = { optional = true }
开启方式	<code>cargo build --features verbose</code> ;依赖方用 <code>features = ["verbose"]</code>

最小必会清单

- 能说出 Rust 属性系统统一了 C 的哪三类机制(预处理/ `__attribute__` /编译选项)
- 能写 `#[cfg]` 平台分支与 debug/release 分支,并解释与 `#ifdef` 的语法检查差异
- 能区分 `#[cfg]` 与 `cfg!()`,并说出各自适用场景
- 能列出常用 derive 组合(Debug/Clone/Copy/PartialEq/Eq/Hash/Default)与选型
- 能用 `#[allow]/#[deny]` 控制 lint,说出与 `-Werror` 的关系
- 能解释 `#[must_use]` 防止哪类 C 常见事故
- 能读懂一个简单的 `#[proc_macro_attribute]` 并说出 `syn/quote` 的角色

- 能声明与使用 Cargo features,包括可选依赖
- 能说出"运行期配置 vs 编译期配置"的选型判据
- 知道 cargo expand 与 clippy 这两个配套工具

自测题

Q 1. #[cfg] 与 #ifdef 最本质的差别是什么?对正确性意味着什么?

✓ 参考答案

处理时机:预处理在语法解析前按文本删减,#ifdef 未选中的分支连语法都不检查,错误潜伏在"没编译到的代码"里(开 DEBUG 才爆);#[cfg] 在语法解析后做条件取舍,所有分支都必须先通过语法解析,只有类型检查与代码生成按条件进行。正确性收益:条件分支里的语法错误、括号不匹配等在编译时必然暴露,不存在"暗藏分支"。

Q 2. 什么时候用 #[cfg] 删条目,什么时候用 cfg!() 留代码?

✓ 参考答案

整个条目(函数/模块/impl)在该条件下无意义→#[cfg](删除后连符号都不存在,体积与检查都更省);一段逻辑仅取值不同、代码量小→cfg!()(代码始终合法,表达式位置随意用)。判据之二:cfg!() 里的代码在两种构建中都必须通过类型检查;若分支两边类型差异大或依赖不同依赖,必须用 #[cfg]。

Q 3. #[allow(dead_code)] 与 #![deny(unsafe_code)] 的层级与作用域有什么区别?

✓ 参考答案

前者是"条目属性"(无 !),只作用于紧随其后的那个条目,局部豁免某条 lint;后者是"内部属性"(带 !),放在 crate 或模块顶部,作用于整个容器。工程形态:全局策略用 #![deny/![warn] 写进 crate 根部或 Cargo.lints;局部豁免用条目级 #[allow]——"默认严格、逐点豁免"是 Rust 的 lint 哲学,与 C 的"默认宽松、逐个开告警"相反。

Q 4. #[must_use] 对应的 C 事故是什么?它的检查在哪个阶段?

✓ 参考答案

对应 C 里"返回值被忽略"的经典事故:忘查 fclose/fflush 失败、strtok 返回 NULL 不判、realloc 失败丢原指针。Rust 把"必须使用返回值"写成类型/函数级契约:调用方忽略带 #[must_use] 的返回值,编译器发出警告(unused_must_use),检查发生在编译期。生产上给"返回 Result/安全相关"的函数统一标注,把"忘查"变成编译警告。

Q 5. 过程宏为什么必须放独立 crate?生产上什么情况下才值得自研?

✓ 参考答案

过程宏以"编译器插件"形态运行:它被编译后由编译器加载执行,与主 crate 的依赖图隔离,因此必须独立成 proc-macro crate(这也是 Rust 团队隔离宏崩溃、保持编译期稳定性的设计)。自研的合理场景:你确实需要"读代码、改代码"的元编程(自动注册、按结构体生成状态机、DSL);若只是加样板,先用 macro_rules!(第10章)与成熟库(thiserror/serde)解决。自研时:保持宏行为可预期、提供展开测试。

Q 6. features 与"运行期配置"如何分工?举一个生产实例。

✅ 参考答案

分工:构建期定"能力集合"(编译哪些代码、链接哪些依赖),运行期定"行为取值"。实例:一个服务支持"数据库存储与内存存储"两种后端——用 feature 决定编译哪个后端模块(编译期,体积与攻击面可控),而端口、日志级别、限流阈值用配置文件(运行期,免重编译)。反例警示:把日志级别做成编译期开关,调级要重编译,运维会恨你——级别永远是运行期配置。

章末实验题:计时属性宏与特性开关

📄 实验 12:计时属性宏 + #[cfg] 开关

实验目标:做一个"带计时能力的程序":① 用 #[cfg] 实现 debug/release 两套日志行为;② 用 Cargo feature 控制"详细模式";③ 实现 #[timed] 属性宏(独立 proc-macro crate),给函数自动计时。

实验要求:

- 任务 1:debug/release 双分支的 log 函数(覆盖:#[cfg] §12.3)
- 任务 2:cfg!() 实现阈值选择(覆盖:cfg!() §12.3)
- 任务 3:Cargo.toml 声明 feature "verbose",代码感知开关(覆盖:features §12.7)
- 任务 4:建立 timer-macros 过程宏 crate,实现 #[timed](覆盖:属性宏 §12.6)
- 任务 5:用 #[timed] 标注 2-3 个函数,验证每次调用输出耗时(覆盖:宏使用)
- 加分:在 #[timed] 基础上加 #[cfg_attr] 控制"release 下自动内联"(覆盖:cfg_attr §12.3)

工程结构:workspace 或两个独立 crate:主程序 + timer-macros(依赖路径 "path = "../timer-macros")。

第一步 一步:cargo new timed_demo 与 cargo new timer-macros --lib;后者 Cargo.toml 加 proc-macro = true

第二步 二步:写 timer-macros 的 #[timed](参考 §12.6 源码),先在本 crate 用 trybuild 前手工验证

第三步 三步:主 crate 依赖 timer-macros,写 2-3 个 #[timed] 函数并调用

第四步 四步:写 #[cfg(debug_assertions)] 双分支日志,分别 cargo run 与 cargo run --release 对比输出

第五步 五步:加 [features] verbose = [],双分支 log_detail,用 --features verbose 验证开关

✅ 参考答案要点

```
// ===== 实验 12:计时属性宏与特性开关 =====
// 覆盖知识点:#[cfg] 条件编译(§12.3)、cfg!()(§12.3)、cfg_attr(§12.3)、
//           features 开关(§12.7)、属性宏(§12.6)

// ---- timer-macros/Cargo.toml ----
// [package] name = "timer-macros" version = "0.1.0" edition = "2021"
// [lib] proc-macro = true
// [dependencies] quote = "1" syn = "2"

// ---- timer-macros/src/lib.rs ----
use proc_macro::TokenStream;
use quote::quote;

/// #[timed]:包裹无参函数并打印耗时(教学简化版)
```

```

#[proc_macro_attribute]
pub fn timed(_attr: TokenStream, item: TokenStream) -> TokenStream {
    let func = syn::parse_macro_input!(item as syn::ItemFn);
    let name = &func.sig.ident;
    let body = &func.block;
    quote! {
        fn #name() {
            let __start = std::time::Instant::now();
            #body
            println!("[计时] {} 耗时 {:?}",
                    stringify!(&name), __start.elapsed());
        }
    }
    .into()
}

// ---- timed_demo/Cargo.toml ----
// [package] name = "timed_demo" version = "0.1.0" edition = "2021"
// [features]
// default = []
// verbose = [] # 详细模式开关
// [dependencies]
// timer-macros = { path = "../timer-macros" }

// ---- timed_demo/src/main.rs ----
use timer_macros::timed;

// 任务 1: debug/release 双分支日志
#[cfg(debug_assertions)]
fn log(msg: &str) {
    println!("[DEBUG] {msg}");
}

#[cfg(not(debug_assertions))]
fn log(msg: &str) {
    println!("[INFO] {msg}");
}

// 任务 2: cfg!() 阈值选择(代码始终存在,值编译期定)
fn threshold() -> u32 {
    if cfg!(debug_assertions) { 10_000 } else { 1_000_000 }
}

// 任务 3: feature 感知的详细日志
#[cfg(feature = "verbose")]
fn log_detail(msg: &str) {
    println!("[详细] {msg}");
}

#[cfg(not(feature = "verbose"))]
fn log_detail(_msg: &str) {
    // 关闭时静默:调用方代码无需改动
}

// 任务 5: 计时宏上场

```

```
#[timed]
fn heavy_sum(n: u32) {
    let mut total = 0u64;
    for i in 1..=n {
        total += i as u64;
    }
    println!("sum(1..={n}) = {total}");
}

#[timed]
fn heavy_fib(n: u32) {
    let mut a = 0u64;
    let mut b = 1u64;
    for _ in 0..n {
        let t = a + b;
        a = b;
        b = t;
    }
    println!("fib({n}) = {a}");
}

fn main() {
    log("程序启动");
    log_detail("进入主流程");
    println!("当前阈值:{}", threshold());

    heavy_sum(100_000);
    heavy_fib(40);
    log("程序结束");
}

// 运行:
// cargo run          → [DEBUG]/[INFO] + 两次 [计时] 输出(无详细日志)
// cargo run --release → [INFO] + 阈值变大 + 耗时明显缩短(内联优化)
// cargo run --features verbose → 多出 [详细] 行
```

设计说明:三个开关各演示一种形态——`#[cfg]` 换"函数存在性"(debug/release 各编译一份)、`cfg!()` 换"常量取值"、`feature` 换"能力开关"。注意 `#[timed]` 宏把原函数体嵌进包裹代码:原函数内定义的变量、`println!` 全部照常工作,这正是"属性宏改写代码"与"普通函数封装"的本质区别。扩展练习:把 `#[timed]` 改成支持带参数函数(需要把 `syn::ItemFn` 的 `sig` 原样搬进生成代码),你会更深入理解过程宏的"代码搬运"本质。

下一章预告:第13章 模块与 Cargo 工程——`crate/mod/use/pub` 的组织方式替代头文件体系;Cargo 全流程、`features`、`workspace` 与 `crates.io` 依赖管理。